Universidad del Valle de Guatemala
CC3067 Redes | Jorge Yass

Estación Meteorológica IoT con Apache Kafka

Telemetría en tiempo real y payload restringido de 3 bytes

Ricardo Godínez - 23247
Vianka Castro - 23201
12 de noviembre de 2026

Qué implementamos



- Tres sensores simulados: temperatura [0, 110.00] °C, humedad [0, 100] % y viento sobre ocho rumbos
- Temperatura y humedad se generan con una distribución normal centrada en 55
- Las dos puntas solo se conocen por el nombre del topic, nunca se hablan directo

Primeras fases: telemetría en JSON

- El productor arma el dato y lo publica con la llave sensor1
- El consumidor se suscribe, acumula el histórico y redibuja con cada mensaje
- Cada mensaje pesaba entre 61 y 63 bytes

```
{"temperatura": 56.32,
 "humedad": 51,
 "direccion_viento": "SO"}
```



Productor y consumidor corriendo contra el servidor del curso.

Fase 3.4: todo en 24 bits

El mensaje no puede pasar de 3 bytes, como en LoRa o Sigfox. Había que recortar unas veinte veces el tamaño sin perder ningún dato.



- Humedad: 101 valores posibles, entran en 7 bits (128)
- Viento: ocho rumbos, entran exactos en 3 bits
- Quedaban 14 bits para la temperatura, que era el campo problemático

Cómo cabe la temperatura en 14 bits

Dejamos de mandarla como flotante y mandamos centésimas de grado.

110.00

valor máximo en grados

11 000

el mismo valor por 100

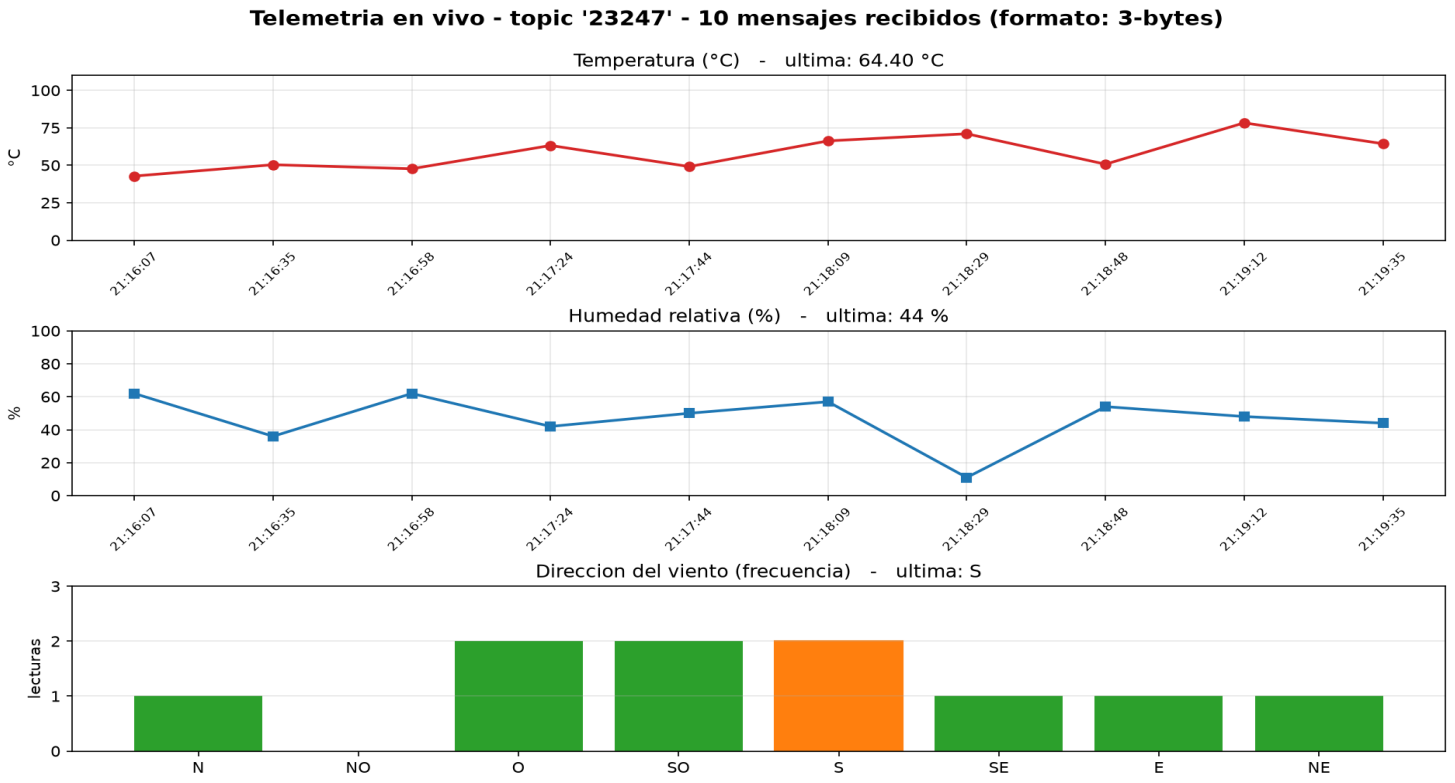
16 384

lo que alcanzan 14 bits

56.32 °C viaja como 5632 y el consumidor divide entre 100 para recuperarlo

El rango completo entra sin recortar nada y sin perder los dos decimales. Encode y decode viven en el mismo módulo para que las dos puntas no se desincronicen.

Evidencia de la corrida con 3 bytes



Telemetría graficada en vivo a partir de mensajes de 3 bytes.

- Diez lecturas enviadas, diez recibidas y decodificadas
- Intervalo real de 15 a 30 segundos
- Mismo resultado que con JSON: los valores rondan 55
- El consumidor detecta el formato por el tamaño del mensaje

El servidor del curso dejó de responder después de la entrega del código, así que esta corrida salió contra un Kafka local con la misma autenticación SASL y el mismo código sin modificar.

Lo que viajó por la red

Bytes (hex)	Temperatura	Humedad	Viento
42e5f0	42.81 °C	62 %	N
4ebd22	50.39 °C	36 %	O
62cd57	63.23 °C	42 %	NE
6f045c	71.05 °C	11 %	S
7a6982	78.34 °C	48 %	O

Los valores que salieron del productor y los que el consumidor reconstruyó coinciden exactamente.

61 a 63 bytes
en JSON

3 bytes
en binario de 24 bits

alrededor de 20 veces menos

Preguntas: el formato de los datos

1. A qué capa pertenece JSON o SOAP según el modelo OSI y por qué

- A la capa 6, la de presentación: define como se representan los datos, no como se transportan
- SOAP queda en la frontera con la capa 7, porque además trae reglas de intercambio
- En el modelo TCP/IP las dos cosas caen dentro de la capa de aplicación

2. Qué beneficios tiene usar un formato como JSON o SOAP

- El mensaje se explica solo, lo que hace mucho más rapido depurar
- Desacopla las dos puntas: agregar un campo no rompe al que ya funcionaba
- Hay librerías en prácticamente cualquier lenguaje
- El costo es el tamaño: los nombres de los campos viajan en cada mensaje

Preguntas: el modelo Pub/Sub de Kafka

3. Ventajas y desventajas del acercamiento Pub/Sub

- A favor: el productor publica y se olvida, no sabe quién lo lee ni si está despierto
- A favor: Kafka guarda los mensajes, varios consumidores leen el mismo flujo y las particiones dan escala
- En contra: hay que operar un broker pesado y no existe respuesta hacia el productor
- En contra: el orden solo se garantiza dentro de una partición y hay que entender los offsets

4. Para qué aplicaciones tiene sentido Kafka y para cuáles no

- Sí: telemetría, bitácoras, métricas, integración entre microservicios, cualquier flujo de eventos con historial
- No: cuando se necesita respuesta inmediata, ahí va una API sobre HTTP
- No: en dispositivos muy limitados, donde encajan mejor MQTT o CoAP
- No: como base de datos para consultar un valor por su llave

Preguntas: trabajar con el payload restringido

5. Qué complejidades introduce un payload pequeño

- El mensaje deja de explicarse solo: hay que decodificar para saber que paso
- El acomodo de los bits se vuelve un contrato, y un bit corrido se lee como basura
- Hay que negociar rango y resolucion de antemano, y no cabe ni versión ni marca de tiempo

6. Como hacemos que la temperatura quepa en 14 bits

- Mandando un entero escalado en vez de un flotante: temperatura por 100
- El máximo pasa a ser 11000 y 14 bits alcanzan para 16384 valores
- El consumidor divide entre 100 y recupera el valor sin perder decimales

Preguntas: si las restricciones aprietan más

7. Qué pasa si la humedad también fuera flotante con un decimal

- Pasaría de 101 a 1001 valores, o sea de 7 a 10 bits, y el total llegaría a 27 bits
- La salida más limpia es bajarle un decimal a la temperatura: 11 + 10 + 3 da exactamente 24 bits
- La otra opción es recortar el rango útil o mandar las variables en mensajes alternados

8. Qué parámetros o herramientas de Kafka ayudarían

- La compresión no sirve sobre 3 bytes: el encabezado del algoritmo pesa más que el dato
- linger.ms y batch.size juntan varias lecturas en un solo envío
- Bajar acks quita confirmaciones; quitar llave y headers recorta metadatos que pesaban más que el payload
- message.max.bytes y las cuotas ponen el techo del lado del broker
- Si el formato tuviera que crecer, Avro o Protobuf con registro de esquemas

Conclusiones

- Metimos las tres lecturas en 3 bytes sin perder nada de lo que pedía el enunciado, y el consumidor siguió graficando igual que con JSON
- El truco estuvo en dejar de mandar la temperatura como flotante: 11000 cabe en 14 bits, un float necesita 32
- La discusión no es cuál formato es mejor, sino que se paga en cada caso: JSON cuesta veinte veces más bytes y regala legibilidad y tolerancia a cambios
- La restricción no vino de Kafka sino de la red de abajo; Kafka solo ofrece herramientas alrededor del problema
- Si lo volviéramos a hacer, reservaríamos un par de bits para versionar el formato